OpenCLI 技術解析報告

以 Ubuntu / Chrome / ChatGPT Web Adapter 開發案例為中心的系統分析

一、前言

本報告的目的，不是單純列出 OpenCLI 的使用指令，而是從實際工程實作的角度，解析 OpenCLI 在本機環境中的執行模型、模組分工、瀏覽器控制路徑，以及如何以它為基礎，擴充出一個可在 Ubuntu 上操作 ChatGPT 網頁版的 CLI Adapter。

這份報告以一個具體案例為核心：原本內建的 `opencli chatgpt` 命令在 Ubuntu 環境中無法達成需求，原因並不只是平台不同，而是其實作路徑本身偏向 macOS Desktop App automation，而不是 ChatGPT Web automation。因此，真正的工程任務不是修補既有命令，而是理解 OpenCLI 的抽象層與執行機制，並基於其 browser/page 能力設計新的 `chatgpt-web` adapter。

本報告將從環境條件、系統架構、程式結構、模組責任、問題分析、修正策略、Codex 輔助開發流程、測試結果與後續工程建議等面向進行完整說明。

二、專案背景與需求定義

在這次任務中，需求可以明確表述如下：

1. 在 Ubuntu / Linux 環境中使用 OpenCLI
2. 依賴 Google Chrome 既有登入狀態
3. 讓 CLI 可以操作 ChatGPT 網頁版，而不是 ChatGPT Desktop App

4. 能以命令列形式執行以下動作：

- 開啟 ChatGPT
- 建立新對話
- 送出 prompt
- 讀取回應
- 回報當前狀態

這類需求本質上屬於「Browser-backed CLI」問題，也就是把一個原本需要在瀏覽器內互動的網站操作流程，轉成可以在命令列中調用的 deterministic interface。

如果從工具定位來看，OpenCLI 本身就是為這類問題而設計的；但問題在於內建的 ChatGPT adapter 並未採用 Browser-backed 路徑，而是採取 OS-level desktop automation。於是，需求與現有命令之間產生了落差。

三、環境盤點與前置條件

要讓 OpenCLI 類型的工具真正可用，前置條件遠比一個 `npm install` 複雜。這次實際工作中整理與驗證完成的環境包括：

- Google Chrome Stable
- Node.js 22
- npm
- OpenCLI
- OpenAI Codex CLI
- OpenCLI daemon

- OpenCLI browser extension

其中最重要的不是安裝本身，而是下列條件必須同時成立：

3.1 Chrome 可作為執行載體

OpenCLI 不是只靠 HTTP request 擷取網站資料。對於互動式網站操作，它依賴真實的瀏覽器 session。因此 Chrome 不只是顯示容器，而是整個 automation path 的核心。

3.2 Browser extension 已正常連線

OpenCLI 需要透過 browser extension 與 daemon 建立 browser bridge。如果 extension 未連線，即便本機安裝了 opencli，很多依賴頁面操作的命令也無法工作。

3.3 Daemon 正常運作

OpenCLI 的 browser operations 背後需要 daemon 提供命令執行橋接，例如頁面導向、evaluate、CDP 指令傳遞等。如果 daemon 未啟動，CLI 無法完成實際 automation。

3.4 Browser session 已登入

對 ChatGPT、X、Bilibili、YouTube 等服務來說，OpenCLI 的優勢之一是直接重用本機瀏覽器既有登入狀態。因此使用者的登入 session 必須已存在於 Chrome 裡。

3.5 驗證方式

這些條件最後透過 `opencli doctor` 完成驗證。doctor 結果顯示：

- daemon 正常

- extension 已連線

- connectivity 正常

這代表 browser bridge 的整條路徑已建立完成，具備開發新 adapter 的基礎。

四、OpenCLI 的系統架構觀察

圖 1：OpenCLI 系統架構圖

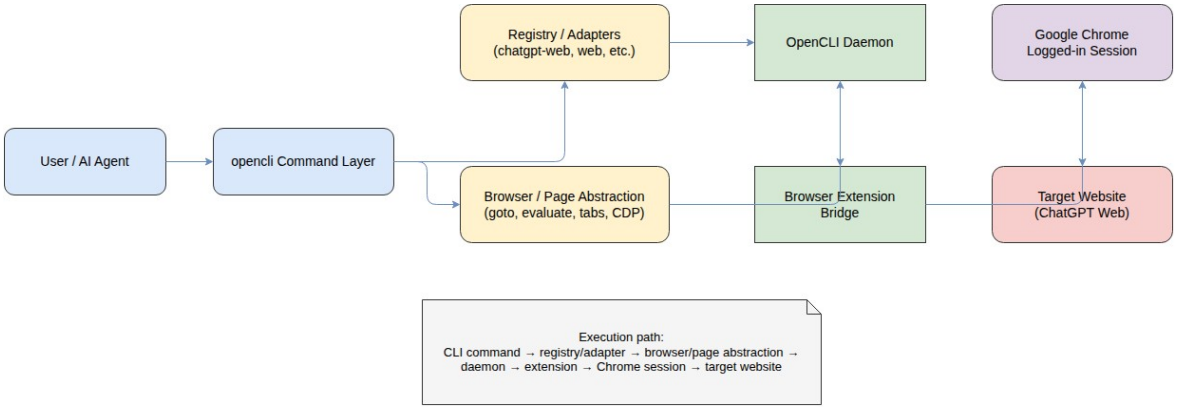


圖 2：內建 chatgpt 與 chatgpt-web adapter 分層比較

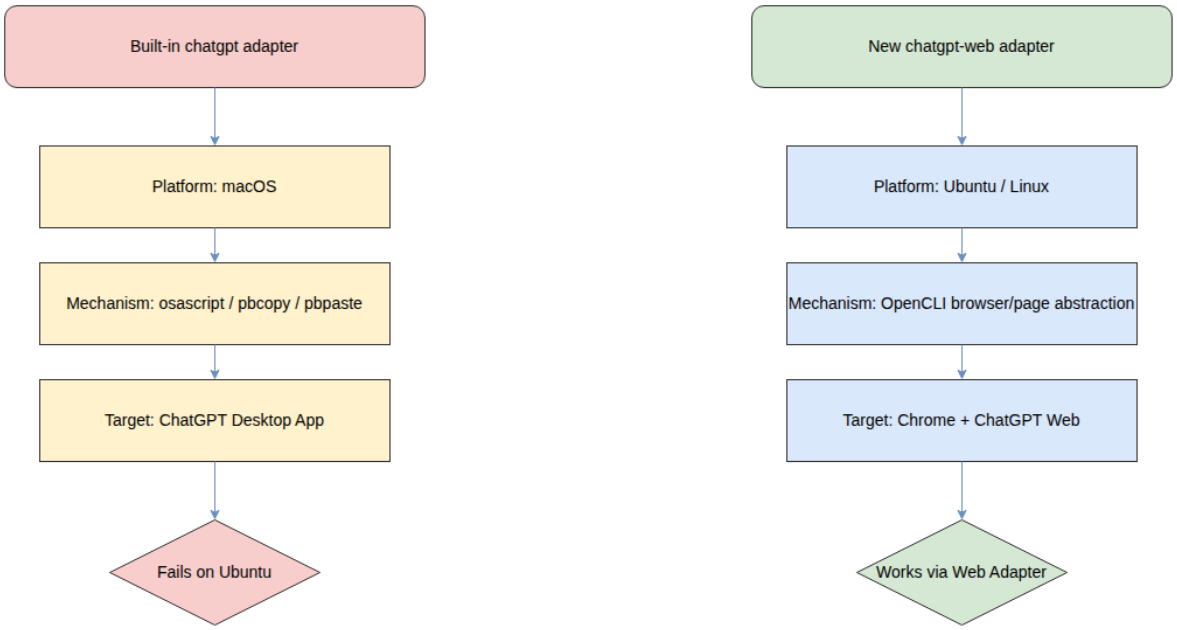


圖 3：chatgpt-web ask 流程圖

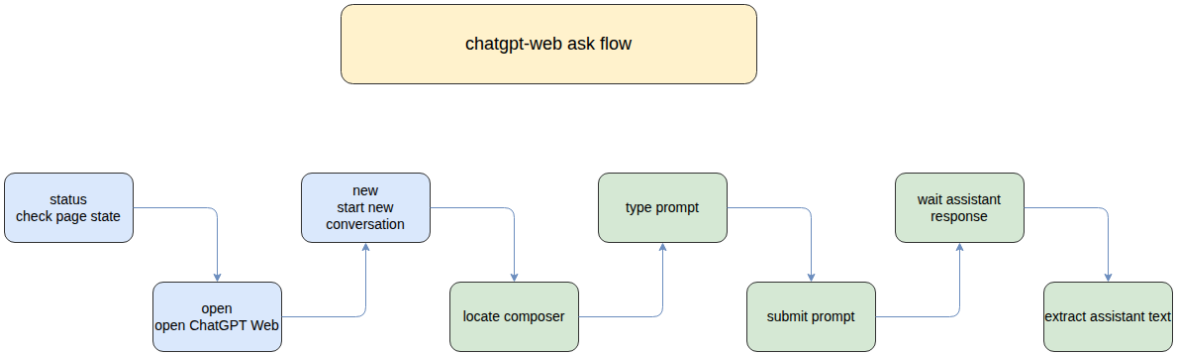
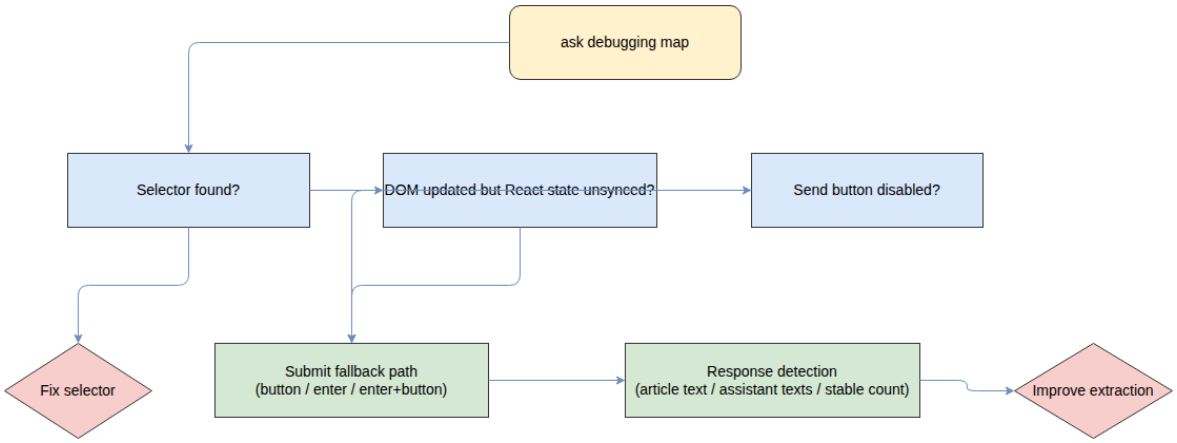


圖 4：ask 除錯問題地圖



從實際安裝結構來看，OpenCLI 的設計有一個很重要的特色：它將「核心執行程式」與「使用者層 adapter / 配置」分離。

4.1 使用者層目錄

使用者層的主要資料位於：

- ~/.opencli/

這裡包含：

- adapter manifest
- clis 目錄
- browser 相關檔案
- 本地插件與額外依賴
- 本機 adapter 的可編輯實作

4.2 套件層目錄

OpenCLI 的核心程式通常位於全域 npm 套件路徑，例如：

- ~/.npm-global/lib/node_modules/@jackwener/opencli/

這裡包含：

- browser abstraction
- registry API
- daemon runtime
- base page / CDP utilities
- 各種內建核心支援模組

4.3 工程上的意義

這種分層讓 OpenCLI 很適合做本地客製化：

1. 使用者可以在 ~/.opencli/clis/ 新增命令
2. 不需要直接改 npm 套件本體
3. 更新 OpenCLI 核心與本地 adapter 可以分開進行
4. 非常適合讓 AI Agent 在本機迭代新 CLI 能力

這一點對本次 `chatgpt-web` adapter 的開發極其重要。因為我們不必修改 OpenCLI 核心，只要新增新的 adapter 即可。

五、Browser / Page 抽象層的角色

OpenCLI 能把網站轉成 CLI，不是靠單純抓 HTML，而是透過 browser/page abstraction。這層能力大致集中在以下模組：

- dist/src/browser/page.js
- dist/src/browser/cdp.js
- dist/src/browser/*

從 `Page` 類別可觀察到的能力包括：

- goto(url)：導向指定頁面
- evaluate(js)：在頁面上下文執行 JavaScript
- tabs() / selectTab()：管理 browser tabs
- screenshot()：擷取頁面畫面
- cdp(...)：透過 CDP 執行更低階控制
- insertText(...)：插入文字
- setFileInput(...)：設定 file input
- network capture / intercept 類能力

5.1 工程上的意義

這表示 OpenCLI 並不是一個單純「網站爬蟲 CLI」。對需要登入、需要互動、需要保持單頁應用狀態的網站來說，它比較像是一層建立在 Chrome session 之上的 automation SDK，再透過 registry 包裝成命令列介面。

換句話說，OpenCLI 的真正優勢不是「抓資料」，而是：

- 重用 session
- 保留前端應用狀態
- 在頁面內執行邏輯
- 把這些能力統一暴露成 CLI

六、Registry 機制與命令定義模式

OpenCLI 的命令通常透過 `cli({...})` 註冊。這裡反映出它的 registry-driven 設計。

典型命令定義包含：

- site
- name
- description
- strategy
- browser
- args
- columns
- func

6.1 對 adapter 開發的意義

這讓新命令不需要侵入 OpenCLI 核心，而只需要：

1. 放到正確位置

2. 正確註冊

3. 使用 browser/page 能力

4. 輸出符合 OpenCLI CLI 風格的結果

對本次任務而言，這就是 `chatgpt-web` 能被自然加入 `opencli list` 的原因。

七、Page 抽象層與控制迴圈的程式解析

圖 A：Page 抽象層三層結構圖

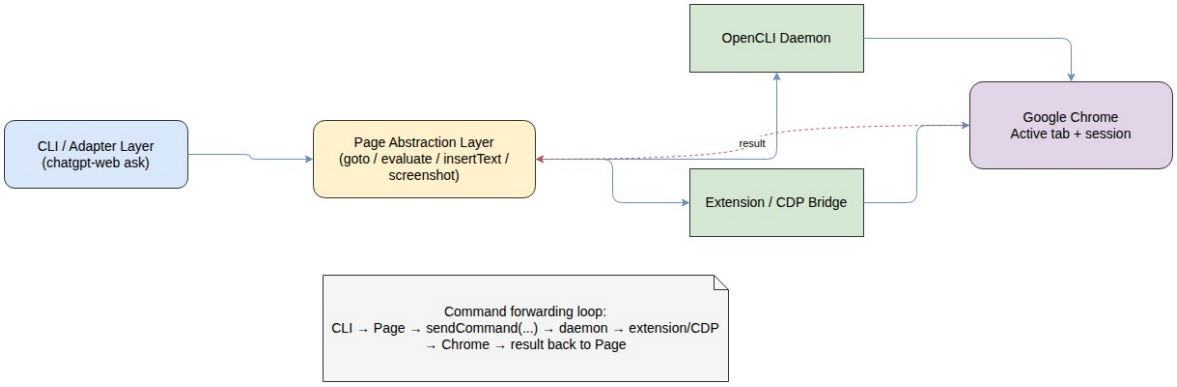
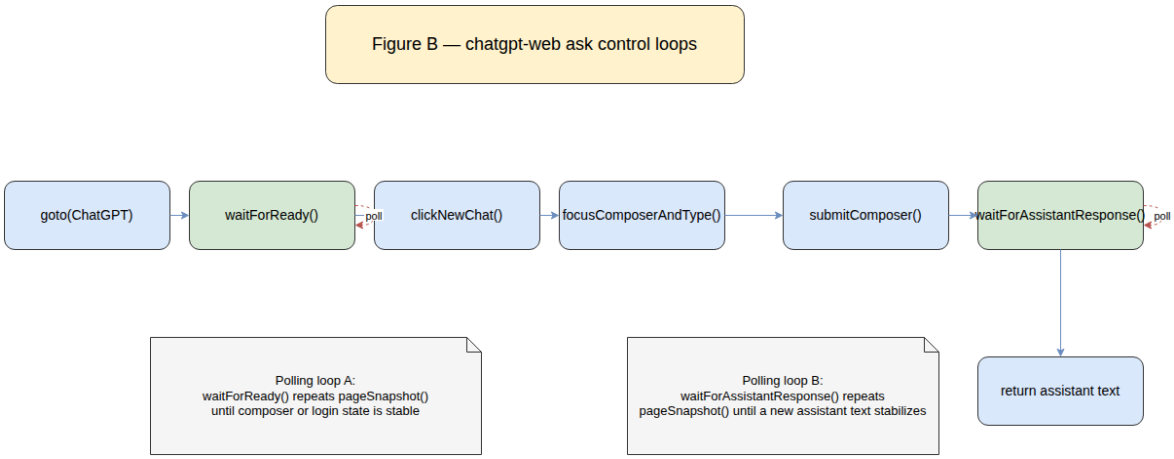


圖 B：chatgpt-web ask 控制迴圈圖



補充章節：chatgpt-web ask 的控制迴圈與程式註解

這一節專門解釋 `chatgpt-web ask` 的流程，目的不是只讓讀者知道它「做了哪些步驟」，而是讓讀者理解：

1. 每一個步驟實際上在做什麼
2. 哪些地方形成了「命令轉發迴圈」
3. 哪些地方形成了「狀態輪詢迴圈」
4. 為什麼這些迴圈對 ChatGPT 這種 SPA 網站是必要的

一、先看高階流程

`chatgpt-web ask` 的高階流程如下：

1. `goto(ChatGPT)`

2. `waitForReady()`

3. `clickNewChat()`

4. `focusComposerAndType()`

5. `submitComposer()`

6. `waitForAssistantResponse()`

7. 回傳 assistant text

如果只看這 7 步，會覺得這像一般自動化腳本，但實際上每一步底下都還有更細的命令流與狀態判斷。

二、第一層迴圈：命令轉發迴圈

OpenCLI 的 `Page` 抽象層，並不是直接控制 Chrome。它的實際模式是：

```
Adapter / CLI
  ↓
Page abstraction
  ↓
sendCommand(...)
  ↓
OpenCLI daemon
  ↓
Browser extension / CDP
  ↓
```

Google Chrome

↓

結果回傳

也就是說，每呼叫一次：

- `goto()`
- `evaluate()`
- `screenshot()`
- `cdp()`
- `insertText()`

最後都會變成一次「命令送出 → daemon 執行 → 結果回傳」的控制迴圈。

這是理解 OpenCLI 的第一個關鍵：

Page API 不是瀏覽器本身，而是瀏覽器控制命令的中介層。

三、程式碼範例 1：goto(url)

以下是 `Page.goto()` 的關鍵邏輯（概念化摘錄）：

```
async goto(url, options) {
    const result = await sendCommand('navigate', {
        url,
        ...this._cmdOpts(),
    });
```

```
    if (result?.tabId) {
        this._tabId = result.tabId;
    }

    this._lastUrl = url;

    if (options?.waitUntil !== 'none') {
        const combinedCode = `${generateStealthJs()}\n$
{waitForDomStableJs(...)}`;
        await sendCommand('exec', { code: combinedCode, ...this._cmdOpts() });
    }
}
```

白話註解

- `sendCommand('navigate', ...)`
  - 把「我要打開某個網址」這件事交給 daemon 去做

- `result?.tabId`
  - 若 Chrome 成功打開頁面，會回傳目前實際使用的 tabId
  - 這個 tabId 會被記住，避免後面的命令跑到錯誤分頁

- `this._lastUrl = url`
  - 記住最後一次打開的是哪個網址

- `sendCommand('exec', ...)`
  - 導頁後，不是立刻就當成功
  - 還會再送一段 JavaScript 到頁面裡檢查 DOM 是否穩定
  - 這相當於「打開頁面之後，再確認頁面是不是可用」

這裡形成的迴圈

這段的迴圈不是 `for/while` 形式，而是：

1. 送出 navigate 命令
2. daemon 執行
3. 回傳 tabId
4. 再送一次 exec 命令
5. 再取得穩定化結果

所以它是「命令串接式控制迴圈」。

四、程式碼範例 2：evaluate(js)

`evaluate()` 幾乎是 OpenCLI browser automation 最核心的方法：

```
async evaluate(js) {
    const code = wrapForEval(js);
    try {
        return await sendCommand('exec', { code, ...this._cmdOpts() });
    } catch (err) {
        if (!isRetryableSettleError(err)) throw err;
        await new Promise((resolve) => setTimeout(resolve, 200));
        return sendCommand('exec', { code, ...this._cmdOpts() });
    }
}
```

白話註解

- `wrapForEval(js)`
  - 把傳進來的 JavaScript 包成 daemon 能理解的格式

- `sendCommand('exec', ...)`
  - 真正讓頁面執行這段 JavaScript

- `catch (err)` + retry
  - 如果頁面剛好在 SPA 切換、target 暫時重建、CDP session 失效
  - 那就等一下再試一次

這裡形成的迴圈

這裡是一個典型的「可重試控制迴圈」：

1. 執行一次
2. 若 target 暫時不穩
3. 等 200ms
4. 再執行一次

所以這是 OpenCLI 對現代 SPA 網站的一種容錯設計。

五、第二層迴圈：狀態輪詢迴圈

上面講的是命令送出層。接下來看 `chatgpt-web ask` 自己的流程。

例如：

```
1. goto(ChatGPT)
2. waitForReady()
3. clickNewChat()
4. focusComposerAndType()
5. submitComposer()
6. waitForAssistantResponse()
7. return assistant text
```

真正形成輪詢的是：

- `waitForReady()`
- `waitForAssistantResponse()`

因為它們都不是「執行一次就完成」，而是：

1. 呼叫 `page.evaluate(...)` 讀當前頁面狀態
2. 判斷條件是否成立
3. 若不成立就等待一小段時間
4. 再呼叫一次 `page.evaluate(...)`
5. 直到條件成立或 timeout

這就是第二層「狀態輪詢迴圈」。

六、程式碼範例 3：waitForReady()

概念化後的寫法如下：

```
async function waitForReady(page, timeoutSeconds = 20) {
  for (let i = 0; i < timeoutSeconds; i++) {
    const snap = await pageSnapshot(page);
    if (snap.composer || snap.loginMarkers) return snap;
    await page.wait(1);
  }
  return pageSnapshot(page);
}
```

白話註解

- 每秒檢查一次頁面
- 看輸入框（composer）是不是已經出現
- 或看登入標記是否出現
- 若頁面還沒穩定，就再等一秒

為什麼這是必要的？

因為 ChatGPT 網頁版不是靜態 HTML。剛進頁面時：

- DOM 可能還沒載完
- React component 可能還沒 mount 完成

- selector 一開始可能抓不到東西

所以如果不輪詢，只用一次 `querySelector()`，很容易誤判「沒有輸入框」。

七、程式碼範例 4：waitForAssistantResponse()

概念化後的寫法如下：

```
async function waitForAssistantResponse(page, beforeTexts, timeoutSeconds = 60)
{
  let lastText = '';
  for (let i = 0; i < timeoutSeconds; i++) {
    const snap = await pageSnapshot(page);
    const fresh = snap.assistantTexts.filter(t => !beforeTexts.includes(t));

    if (fresh.length) {
      const latest = fresh[fresh.length - 1];
      if (latest === lastText) {
        return latest;
      }
      lastText = latest;
    }

    await page.wait(1);
  }

  return lastText;
}
```

白話註解

- 每秒讀一次目前畫面上的 assistant texts

- 只找「這次 ask 之後新出現的文字」

- 如果最新那段文字已經連續穩定，就視為回應完成

- 如果一直沒有新文字，就等到 timeout

為什麼不能只讀最後一段文字？

因為 ChatGPT 是動態生成內容的：

- 一開始可能是空白 placeholder
- 中途可能只有半句
- 畫面上還可能殘留舊對話內容

所以如果不做輪詢與比對，就很容易抓錯結果。

八、把兩種迴圈合起來看

整個 `chatgpt-web ask` 其實同時包含兩層迴圈：

第一層：命令轉發迴圈

CLI → Page → daemon → extension / CDP → Chrome → 回傳結果

這層負責把高階需求真正送進瀏覽器。

第二層：狀態輪詢迴圈

adapter 反覆呼叫 `evaluate()` / `pageSnapshot()`，直到頁面狀態符合預期。

這層負責等待 SPA 網站真正進入可用狀態。

如果少了第一層，就無法控制瀏覽器。

如果少了第二層，就無法穩定控制現代前端網站。

九、對讀者最重要的理解點

如果要用一句話總結這段程式：

OpenCLI 的 Page 抽象層，並不是在「直接操作 DOM」，而是在建立一個可重試、可輪詢、可透過 daemon 與 Chrome 溝通的控制迴圈。

因此，`chatgpt-web ask` 之所以難，不是因為少一個 selector，而是因為它同時要處理：

- 正確 tab
- 正確頁面
- React state
- submit 行為
- 回應生成過程
- 最後結果萃取

而這些，都建立在 `Page` 這層控制抽象之上。

十、建議放進文件的圖示對應

這一節最適合搭配兩張圖：

圖 A：Page 抽象層三層結構圖

- Adapter / CLI
- Page abstraction
- daemon / extension / Chrome

圖 B：chatgpt-web ask 迴圈圖

- goto
- waitForReady
- clickNewChat
- focusComposerAndType
- submitComposer
- waitForAssistantResponse
- return assistant text

這樣讀者就能同時看到：

- 程式碼片段
- 中文逐段解釋
- 控制流程圖
- 哪裡形成迴圈

教材化補強章節：如何閱讀 OpenCLI 的控制迴圈

這一章不只是補充技術細節，而是把原本偏工程師視角的說明，改寫成更像教學講義的形式。目標是讓讀者不需要先熟悉 OpenCLI 原始碼，也能理解：

1. 這段程式碼在整體系統裡的位置
2. 它為什麼會形成「迴圈」
3. 這些迴圈如何真的影響 `chatgpt-web ask`
4. 遇到問題時應該先看哪一層

一、先不要急著看程式碼

很多人看到 `goto()`、`evaluate()`、`sendCommand()` 這些函式時，第一反應會是：「這不就是一些工具函式嗎？」

但在 OpenCLI 裡，它們不是普通 helper，而是整個 browser automation 的控制節點。

你可以把 OpenCLI 想成三層：

第一層：你下命令

- 例如：`opencli chatgpt-web ask "今天天氣如何？"`

第二層：Page 抽象層負責翻譯命令

- 例如：`goto()`、`evaluate()`、`insertText()`

第三層：daemon 與 Chrome 真正執行

- daemon 收到命令
- extension / CDP 接手
- Chrome 頁面實際被控制

所以，**Page 不是瀏覽器本身，而是 CLI 與瀏覽器之間的翻譯層。**

二、先用一張腦中地圖理解

請把整體流程先想成下面這張腦中圖：

CLI 命令

↓

Adapter

↓

Page abstraction

↓

sendCommand(...)

↓

Daemon

↓

Extension / CDP

↓

Chrome

↓

頁面結果回傳

這張圖最重要的意義是：

- Adapter 不直接碰 Chrome
- Page 也不直接碰 Chrome
- 真正執行的是 daemon + extension / CDP
- 但 adapter 與 Page 決定了要執行什麼、怎麼輪詢、何時停止

三、教材式閱讀法：先看角色，再看程式

如果直接把原始碼貼上來，很多讀者會立刻迷失在細節裡。

比較好的閱讀方式是：

先問：這段程式碼的角色是什麼？

以 `goto(url)` 為例，它的角色不是「打開網頁」這麼簡單，而是：

- 把導頁需求送給 daemon

- 取得目前正確的 tabId

- 確保後續命令都打到同一個分頁

- 在必要時等待頁面穩定

所以你在讀 `goto()` 時，不要只看「它調了哪些 API」，要看它在**控制鏈**裡扮演什麼角色。

四、範例一：如何讀 `goto()`

原始概念：

```
async goto(url, options) {
    const result = await sendCommand('navigate', {
        url,
        ...this._cmdOpts(),
    });

    if (result?.tabId) {
        this._tabId = result.tabId;
    }

    this._lastUrl = url;

    if (options?.waitUntil !== 'none') {
        await sendCommand('exec', { code: combinedCode, ...this._cmdOpts() });
    }
}
```

先不要急著逐行翻譯

先看它在做哪四件事：

1. 把導頁命令送出去

2. 記住現在真正操作的是哪個 tab

3. 記住最後一次打開的 URL

4. 視需要檢查頁面是否已穩定

再逐段看

`sendCommand('navigate', ...)`

- 白話：請 daemon 幫我打開這個網址

`if (result?.tabId) { this._tabId = result.tabId; }`

- 白話：Chrome 告訴我剛剛其實打開的是哪個分頁，我要把它記下來

- 教學重點：如果不記這個 tabId，後面很容易命令送錯頁

`this._lastUrl = url;`

- 白話：記住最後開過哪個網址

- 教學重點：這不是最核心，但對 debug 很有幫助

`sendCommand('exec', ...)`

- 白話：再送一段程式到頁面裡，確認頁面狀態穩定

- 教學重點：OpenCLI 不把「頁面打開」等同於「頁面可操作」

這一段的教材重點

請記住一句話：

`goto()` 不只是導頁，它其實在建立「後續所有操作應該落在哪個 tab、在哪個狀態下繼續執行」的基礎。

五、範例二：如何讀 `evaluate()`

原始概念：

```
async evaluate(js) {
    const code = wrapForEval(js);
    try {
        return await sendCommand('exec', { code, ...this._cmdOpts() });
    } catch (err) {
        if (!isRetryableSettleError(err)) throw err;
        await new Promise((resolve) => setTimeout(resolve, 200));
        return sendCommand('exec', { code, ...this._cmdOpts() });
    }
}
```

這段在做什麼？

不要把它只看成「執行 JavaScript」。

它真正的角色是：

- 把頁面端程式碼包裝好
- 透過 daemon 丟進正確 tab
- 如果頁面 target 暫時失效，就等一下再重試

教材式拆法

`wrapForEval(js)`

- 白話：把你要在頁面裡跑的程式碼包裝成可執行形式

`sendCommand('exec', ...)`

- 白話：請 daemon 到 Chrome 裡幫我執行這段 code

`catch (err)` + retry

- 白話：如果頁面正在切換或 SPA target 暫時不穩，不要立刻放棄，再試一次

教學重點

這說明了 OpenCLI 的設計假設：

現代網頁不穩定是常態，不是例外。

所以 `evaluate()` 的核心不是「幫你跑 JS」，而是「幫你在不穩定頁面裡，盡量可靠地跑 JS」。

六、兩種迴圈一定要分清楚

讀這份文件時，最容易混淆的就是「到底哪裡叫迴圈？」

這裡其實有兩種不同層級：

迴圈 A：命令轉發迴圈

這是這樣的：

CLI → Page → daemon → extension / CDP → Chrome → 回傳結果

每次 `goto()`、`evaluate()`、`screenshot()` 都會走一輪。

這不是 `for` 或 `while` 的程式迴圈，而是控制流程上的閉環。

迴圈 B：狀態輪詢迴圈

這才是你在 code 裡會明顯看到的：

- 讀狀態
- 檢查條件
- 等一下
- 再讀一次
- 直到成功或 timeout

例如：

- `waitForReady()`

- `waitForAssistantResponse()`

最重要的教學句

命令轉發迴圈決定命令送不送得進去，狀態輪詢迴圈決定結果抓不抓得回來。

七、為什麼 `chatgpt-web ask` 特別依賴這兩種迴圈

ChatGPT Web 不是靜態頁面，它有很多動態行為：

- React state 改變

- textarea 值更新

- send button enable/disable

- assistant 回應逐步生成

- 對話內容持續變化

這意味著：

只送一次命令，不夠。

只讀一次 DOM，也不夠。

所以 `ask` 才會長成這種形式：

1. `goto(ChatGPT)`

2. `waitForReady()`

3. `clickNewChat()`

4. `focusComposerAndType()`

5. `submitComposer()`

6. `waitForAssistantResponse()`

7. 回傳結果

教學重點

這不是「流程很多」，而是因為每一段都在處理不同的不確定性：

- 頁面是否已準備好

- 輸入框是否真的存在

- prompt 是否真的進入 React state

- submit 是否真的觸發

- assistant 是否真的開始生成

- 什麼時候算生成完成

八、常見誤解

誤解 1：只要 selector 對了就會成功

不對。selector 對了，只代表你找到了元素，不代表前端 state 真的更新。

誤解 2：DOM value 改了就算輸入成功

不對。React / SPA 網頁常常需要更完整的事件鏈。

誤解 3：只要讀最後一段文字就是答案

不對。最後一段可能是舊內容、空白 placeholder，或尚未穩定的半句。

誤解 4：Page 是瀏覽器本身

不對。Page 是控制抽象層，真正執行命令的是 daemon 與 Chrome。

九、如果讀者只記一件事

如果整段看完只記一句話，最應該記這句：

OpenCLI 的關鍵，不在於它有多少 helper function，而在於它把「命令送進瀏覽器」與「等待頁面進入正確狀態」這兩件事，做成可反覆執行的控制迴圈。

這也是為什麼 `chatgpt-web ask` 能成立的真正原因。

你特別指出的那段內容，真正的重點不是「Page 有哪些 API」，而是這些 API 如何構成一個從 CLI 到瀏覽器再回傳結果的控制迴圈。

若只看功能列表，例如：

- goto(url)

- evaluate(js)

- tabs() / selectTab()

- screenshot()

- cdp(...)

- insertText(...)

- setFileInput(...)

會讓人以為這只是一般工具函式集合。但實際上，Page 是整個 OpenCLI browser automation 的中介層。上層 adapter 每做一個動作，最後都會經過 Page 封裝成 command，再送往 daemon，由 daemon 轉給 browser extension 或 CDP，最後才真的作用在 Chrome 頁面上。

7.1 核心程式碼摘錄

以下是 `dist/src/browser/page.js` 中最關鍵的片段（節錄）：

```
async goto(url, options) {
    const result = await sendCommand('navigate', {
        url,
        ...this._cmdOpts(),
    });

    if (result?.tabId) {
        this._tabId = result.tabId;
    }
    this._lastUrl = url;

    if (options?.waitUntil !== 'none') {
        const maxMs = options?.settleMs ?? 1000;
        const combinedCode = `${generateStealthJs()};\n$
{waitForDomStableJs(maxMs, Math.min(500, maxMs))}`;
        const combinedOpts = {
            code: combinedCode,
            ...this._cmdOpts(),
        };
        try {
```

```
            await sendCommand('exec', combinedOpts);
        } catch (err) {
            if (!isRetryableSettleError(err)) throw err;
            await new Promise((r) => setTimeout(r, 200));
            await sendCommand('exec', combinedOpts);
        }
    }
}

async evaluate(js) {
    const code = wrapForEval(js);
    try {
        return await sendCommand('exec', { code, ...this._cmdOpts() });
    } catch (err) {
        if (!isRetryableSettleError(err)) throw err;
        await new Promise((resolve) => setTimeout(resolve, 200));
        return sendCommand('exec', { code, ...this._cmdOpts() });
    }
}
```

7.2 這段程式碼的結構意義

這段程式的核心可以拆成三個概念：

(1) Page 不直接控制 Chrome

Page 本身沒有直接碰到 Chrome DOM，也沒有自己維護瀏覽器連線。它做的事情是把高階操作翻譯成 `sendCommand(...)`。

(2) 每次操作都帶著上下文

`this._cmdOpts()` 會把 workspace 與目前 tabId 帶進去。這很重要，因為若沒有 tabId，extension 可能會把命令送到錯誤的分頁，例如 `chrome://` 或 extension 頁面。

(3) OpenCLI 的 browser 控制不是一次性，而是可重試的

不論是 `goto()` 後的 settle，還是 `evaluate()` 本身，都有 retryable error 處理。這反映出 OpenCLI 把 SPA 頁面切換、target 重新建立、CDP session 短暫失效，視為常態而不是例外。

7.3 `goto(url)` 的控制迴圈

`goto(url)` 不是單純導頁。它的完整流程其實是：

1. adapter 呼叫 `page.goto(url)`
2. `Page` 呼叫 `sendCommand('navigate', ...)`
3. daemon 接到 navigate 命令
4. daemon / extension / CDP 讓 Chrome 導向該網址
5. Chrome 回傳結果，其中包含 tabId
6. `Page` 記住 tabId，之後所有操作都綁到同一個 tab
7. 若要求等待穩定，接著再送一次 `exec`，在頁面內注入 stealth 與 DOM settle 檢查
8. 如果發現 target 短暫失效，等待 200ms 再重試一次

這就是第一層「請求—執行—回傳—再穩定化」迴圈。

7.4 `evaluate(js)` 的控制迴圈

`evaluate(js)` 是第二個核心控制點。它表面上看起來只是執行一段 JavaScript，但實際流程是：

1. adapter 傳入一段頁面端 JavaScript

2. `wrapForEval(js)` 先把程式碼包裝成 daemon 可執行格式

3. `sendCommand('exec', ...)` 把程式送到 daemon

4. daemon 經由 browser bridge / CDP 在指定 tab 執行

5. 將執行結果回傳給 `Page`

6. 若碰到可重試的 target error，等待後重送一次

因此在工程上，`evaluate()` 其實是 OpenCLI 幾乎所有 DOM 讀取與狀態判定的基礎。

7.5 這些 API 如何形成真正的「程式迴圈」

如果只看單一函式，還不容易理解迴圈。但把它放回 adapter 執行流程裡就很清楚。

例如 `chatgpt-web ask` 的高階流程是：

1. `goto(ChatGPT)`

2. `waitForReady()`

3. `clickNewChat()`

4. `focusComposerAndType()`

5. `submitComposer()`

6. `waitForAssistantResponse()`

7. 回傳 assistant text

而其中的 `waitForReady()` 與 `waitForAssistantResponse()` 並不是一次完成，而是典型的 polling loop：

- 呼叫 `page.evaluate(...)` 讀頁面狀態
- 檢查條件是否成立
- 若未成立就 `wait(1)`
- 再呼叫一次 `page.evaluate(...)`
- 重複直到條件成立或 timeout

所以這裡至少有兩種層次的迴圈：

(1) 命令轉發迴圈

CLI → Page → daemon → extension / CDP → Chrome → 結果回傳

(2) 狀態輪詢迴圈

adapter 反覆呼叫 `evaluate()`，直到頁面進入預期狀態

7.6 為什麼這對 ChatGPT Web 特別重要

像 ChatGPT 這種 React / SPA 網站，不是傳統靜態頁面。它的狀態變化通常不是單一 request 完成，而是：

- 前端渲染更新
- 內部 state 改變

- 按鈕 enable/disable 切換

- assistant response 漸進生成

因此若沒有這種「控制命令迴圈 + 狀態輪詢迴圈」的結構，就很難用 CLI 可靠控制它。

7.7 對 `chatgpt-web` adapter 的直接影響

這段 Page 程式碼的重要性在於，它決定了 adapter 並不是直接操作 DOM，而是站在一個更高的中介層上運作。這讓 adapter 能：

- 在正確 tab 內執行

- 在 SPA 頁面切換時保有一定容錯能力

- 重用 daemon 與 browser bridge 能力

- 透過 `evaluate()` 實作 debug、selector 檢查、response 抓取

也因此，當 `ask` 出問題時，真正要查的不只是 DOM selector，而是：

- 命令有沒有送到正確 tab

- evaluate 是否在正確頁面執行

- 事件是否真的改變前端 state

- response polling 是否判斷對了新的 assistant text

八、為什麼內建 chatgpt 命令在 Ubuntu 上不可用

實際檢查 ~/.opencli/clis/chatgpt/ask.js、send.js、read.js 後，可以清楚看到這個 adapter 的設計假設是：

- 平台是 macOS
- 目標是 ChatGPT Desktop App
- 透過 `osascript` 激活應用程式
- 透過 `pbcopy/pbpaste` 處理 clipboard
- 用 OS-level keystroke 模擬貼上與送出

這種設計在 macOS 是合理的，但在 Ubuntu 上就會直接失效。錯誤訊息如：

- osascript: not found
- pbpaste: not found

因此問題並不是 OpenCLI 壞了，而是現有 adapter 的平台假設不成立。

八、chatgpt-web Adapter 的設計原則

針對上述落差，本次建立了新的 `chatgpt-web` adapter，其設計原則如下：

1. 明確鎖定 ChatGPT Web，而非 Desktop App
2. 明確鎖定 Ubuntu / Linux + Chrome 環境

3. 完全使用 OpenCLI 的 browser/page infrastructure

4. 不使用任何 macOS-only automation API

5. 將功能拆成多個可獨立驗證的命令

8.1 命令設計

規劃出的命令包括：

- opencli chatgpt-web status
- opencli chatgpt-web open
- opencli chatgpt-web new
- opencli chatgpt-web ask "..."
- opencli chatgpt-web read
- opencli chatgpt-web debug

這不是任意拆分，而是工程上刻意分層：

- status 解決「頁面是否可用」
- open 解決「能不能開頁」
- new 解決「能不能建立新上下文」
- debug 解決「selector / DOM 狀態是否正確」
- ask 解決「真正互動是否成功」
- read 解決「如何穩定讀最後回答」

九、核心函式解析

9.1 ensureChatGPT(page)

- 開啟 https://chatgpt.com/

- 等待頁面有基本穩定性

9.2 pageSnapshot(page)

- 收集 URL、title、composer、send button、login markers、article texts 等狀態

- 這是整個 adapter 的核心觀測點

9.3 waitForReady(page)

- 等待 composer 或 login markers 出現

- 避免在頁面尚未穩定時誤判

9.4 clickNewChat(page)

- 嘗試建立新對話

- 降低舊內容對 ask/read 的干擾

9.5 focusComposerAndType(page, text)

- 找到 textarea / composer

- 嘗試用 native setter 與 input/change event 將 prompt 寫入

- 這段的難點在於 React state 不一定會因為 DOM 改變而同步更新

9.6 submitComposer(page)

- 優先點 send button

- 若按鈕不可用則送 Enter

- 必要時再補一次 button click

9.7 waitForAssistantResponse(...)

- 等待新的 assistant 回應

- 避免抓到舊內容、空白 placeholder 或尚未穩定的文字

十、Debug 過程中的重要發現

1. `status` 成功

   - adapter 已載入

   - ChatGPT 網頁可開啟

   - composer 可找到

2. `debug` 顯示 `TEXTAREA`

   - selector 並非完全錯誤

   - 問題更可能在輸入與送出事件

3. `new` 成功

   - 新對話路徑可達

   - 有助於隔離上下文

4. `ask` 初期回空

   - 流程有跑完

   - 但 ChatGPT 沒有回有效結果

   - 問題集中在 React state、submit、response detection

5. 最終 `ask` 成功取得非空回應

   - 表示 OpenCLI → Chrome → ChatGPT Web → 回應擷取 這條鏈已打通

十一、Codex 在開發流程中的角色

這次工作不是完全交給 Codex 從零完成，而是採取比較成熟的 AI 協作模式：

1. 先由人工建立問題框架與初版實作

2. 先完成環境安裝與工具接通

3. 先做第一輪 debug，把問題縮到 `ask` 的最後一公里

4. 再把問題整理成 `CODEX_TASK.md`

5. 讓 Codex 專注修 `ask` 的最後關鍵段落

這樣的流程有幾個優點：

- AI 不需要猜問題背景

- 需求與限制明確

- Codex 的輸出集中在高價值區域

- 人仍然保有架構與品質控制權

十二、當前成果評估

已達成：

- OpenCLI 在 Ubuntu 上正常運作

- Chrome / Node / npm / Codex 均已安裝並驗證

- `chatgpt-web` adapter 已成功掛入 OpenCLI

- `ask` 已能取得非空 assistant response

- 專案文件、技術報告、PDF 已整理完成

尚未完全完成：

- `read` 仍不夠穩定

- 尚未做完整 conversation/history 設計

- 尚未做 model management

- 尚未做附件上傳支援

- 尚未做正式獨立 plugin packaging

十三、使用建議與工程建議

13.1 實務操作建議

若目前要實際使用，建議流程為：

1. opencli chatgpt-web status

2. opencli chatgpt-web new

3. opencli chatgpt-web ask "..."

在 `read` 完全穩定前，優先信任 `ask` 的直接回傳結果。

13.2 工程演進建議

後續若要把它提升到更正式的生產等級，建議優先做：

1. 強化 `read` selector

2. 補 model switch

3. 補 history / conversation API

4. 補附件上傳

5. 建立 regression tests

6. 整理成獨立 plugin project

7. 提供更正式的 adapter manifest 與安裝路徑

十四、結論

這次案例很有代表性，因為它展示了幾個重要結論：

第一，OpenCLI 的核心能力並不侷限於內建命令本身，而是其 browser/page abstraction 與 registry 架構。只要理解這層抽象，許多原本不存在的 CLI 都可以被建構出來。

第二，當現有 adapter 與需求不相符時，真正的解法常常不是 patch 原始命令，而是重新辨識它的執行模型，然後沿著正確的抽象層重建命令。

第三，Ubuntu + Chrome + ChatGPT Web 的 CLI 路線是可行的。雖然其中最困難的部分在於 React/SPA 的輸入與提交行為，但透過 page-level automation、fallback submit 策略與 response detection 收斂，實際上已經可以得到可用成果。

第四，Codex 適合作為局部工程問題的加速器，而不是整體架構思考的替代品。當人先把問題拆解清楚，再交由 Codex 補最關鍵的一段，其效果是很高的。

總結而言，本次工作已經成功把 OpenCLI 在 Ubuntu 上的 ChatGPT 能力，從原本不適用的平台路線，推進到一個可實際使用的 ChatGPT Web CLI 原型，並且已具備後續演進為正式 plugin 的基礎。